Algoritmos e Estruturas de Dados

Ordenação Interna - Bolha, Seleção e Inserção

CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA DE MINAS GERAIS
Departamento de Computação

Introdução - Conceitos Básicos

- Ordenar: processo de **rearranjar** um conjunto de objetos em uma ordem ascendente ou descendente.
- A ordenação visa facilitar a **recuperação** posterior de itens do conjunto ordenado.
  - Dificuldade de se utilizar um catálogo telefônico se os nomes das pessoas não estivessem listados em ordem alfabética.
- Notação utilizada nos algoritmos:
  - Os algoritmos trabalham sobre os **registros** de um arquivo.
  - Cada registro possui uma **chave** utilizada para controlar a ordenação.
  - Podem existir **outros componentes** em um registro

Introdução - Conceitos Básicos

Estrutura de um registro

```
1 public class Item {
2   private int chave;
3   public Item(int chave) {
4     this.chave = chave;
5     /*outros componentes */
6 }
```

- Qualquer tipo de chave sobre o qual exista uma **regra de ordenação bem-definida** pode ser utilizado.
- Um método de ordenação é **estável** se a ordem relativa dos itens com chaves iguais não se altera durante a ordenação.
- Alguns dos métodos de ordenação mais eficientes não são estáveis.
- A estabilidade pode ser forçada quando o método é não-estável.

Introdução - Conceitos Básicos

- Vetor Não Ordenado:

  [1 2 3 9 5 4 2 2]

- Vetor Ordenado (Estável):

  [1 2 2 2 3 4 5 9]

- Vetor Ordenado (Não Estável):

  [1 2 2 2 3 4 5 9]

Introdução - Conceitos Básicos

- Classificação dos métodos de ordenação:
  - Interna: arquivo a ser ordenado cabe todo na memória principal.
  - Externa: arquivo a ser ordenado não cabe na memória principal.
- Diferenças entre os métodos:
  - Em um método de ordenação interna, qualquer registro pode ser imediatamente acessado.
  - Em um método de ordenação externa, os registros são acessados sequencialmente ou em grandes blocos.
- A maioria dos métodos de ordenação é baseada em **comparações** das chaves.

Ordenação Interna

- Na escolha de um algoritmo de ordenação interna deve ser considerado o tempo gasto pela ordenação.
- Sendo n o número registros no arquivo, as medidas de complexidade relevantes são:
  - Número de comparações $C(n)$ entre chaves.
  - Número de movimentações $M(n)$ de itens do arquivo.
- O uso econômico da memória disponível é um requisito primordial na ordenação interna.
- Métodos de ordenação que utilizam a estrutura de um vetor e executam a permutação dos itens no próprio vetor são os preferidos.
- Métodos que utilizam listas encadeadas não são muito utilizados.
- Métodos que fazem cópias dos itens a serem ordenados possuem menor importância

Classificação dos métodos de ordenação interna

- Métodos simples:
  - Adequados para pequenos arquivos.
  - Requerem (n^2) comparações (onde n é a quantidade de elementos).
  - Produzem programas pequenos.
- Métodos eficientes:
  - Adequados para arquivos maiores.
  - Requerem ($n \times log\ n$) comparações.
  - Usam menos comparações.
  - As comparações são mais complexas nos detalhes.
  - Métodos simples são mais eficientes para pequenos arquivos.

Ordenação Interna

- Tipos de dados e variáveis utilizados nos algoritmos de ordenação interna
- O índice do vetor vai de 0 até *MaxTam*, devido às chaves sentinelas
- O vetor a ser ordenado contém chaves nas posições de 1 até *n*

Ordenação Bolha - *Bubble Sort*

- O algoritmo de ordenação bolha é simples de entendimento e de fácil implementação.
- Está entre os mais conhecidos e difundidos métodos de ordenação de arranjos, mas não é um algoritmo eficiente.
- O princípio do bolha é a troca de valores entre posições consecutivas, fazendo com que os valores mais altos (ou mais baixos) "borbulhem"para o final do arranjo (por isso o nome *Bubblesort*).

Ordenação Bolha - *Bubble Sort*

- A ideia é comparar dois elementos e trocá-los de posição, até que os elementos de maior valor sejam levados para o final do vetor.
- O processo continua até a ordenação total do vetor lembrando a forma como as bolhas em um tanque de água procuram seu próprio nível.
- O tempo (custo) computacional desse algoritmo é de ordem quadrática (n^2)
- Tempo computacional: não se refere ao tempo de relógio e sim ao número de operações (comparações ou trocas) realizadas pelo algoritmo no processo de ordenação em função da quantidade n de itens de entrada.
- Ele não é recomendado para programas que precisem de velocidade e operem com quantidade elevada de dados.

Ordenação Bolha - *Bubble Sort*

i = 1

j		1	2	3	4	5	
1	[	3	1	5	4	2	]

Ordenação Bolha - *Bubble Sort*

i = 1

j		1	2	3	4	5	
1	[	$\underline{3}$	$\underline{1}$	5	4	2	]
2	[	1	$\underline{3}$	$\underline{5}$	4	2	]

Ordenação Bolha - *Bubble Sort*

i = 1

j		1	2	3	4	5	
1	[	$\underline{3}$	$\underline{1}$	5	4	2	]
2	[	1	$\underline{3}$	$\underline{5}$	4	2	]
3	[	1	3	$\underline{5}$	$\underline{4}$	2	]

Ordenação Bolha - *Bubble Sort*

i = 1

j		1	2	3	4	5	
1	[	$\underline{3}$	$\underline{1}$	5	4	2	]
2	[	1	$\underline{3}$	$\underline{5}$	4	2	]
3	[	1	3	$\underline{5}$	$\underline{4}$	2	]
4	[	1	3	4	$\underline{5}$	$\underline{2}$	]

Ordenação Bolha - *Bubble Sort*

i = 1

j		1	2	3	4	5	
1	[	3	1	5	4	2	]
2	[	1	3	5	4	2	]
3	[	1	3	5	4	2	]
4	[	1	3	4	5	2	]
	[	**1**	**3**	**4**	**2**	**5**	]

Ordenação Bolha - *Bubble Sort*

i = 2

j		1	2	3	4	5	
1	[	1	3	4	2	**5**	]

Ordenação Bolha - *Bubble Sort*

i = 2

j		1	2	3	4	5	
1	[	$\underline{1}$	$\underline{3}$	4	2	**5**	]
2	[	1	$\underline{3}$	$\underline{4}$	2	**5**	]

Ordenação Bolha - *Bubble Sort*

i = 2

j		1	2	3	4	5	
1	[	1	3	4	2	**5**	]
2	[	1	3	4	2	**5**	]
3	[	1	3	4	2	**5**	]

Ordenação Bolha - *Bubble Sort*

i = 2

j		1	2	3	4	5	
1	[	1	3	4	2	**5**	]
2	[	1	3	4	2	**5**	]
3	[	1	3	4	2	**5**	]
	[	**1**	**3**	**2**	**4**	**5**	]

Ordenação Bolha - *Bubble Sort*

i = 3

j		1	2	3	4	5	
1	[	1	3	2	**4**	**5**	]

Ordenação Bolha - *Bubble Sort*

i = 3

j		1	2	3	4	5	
1	[	1	3	2	**4**	**5**	]
2	[	1	3	2	**4**	**5**	]

Ordenação Bolha - *Bubble Sort*

i = 3

j		1	2	3	4	5	
1	[	1	3	2	**4**	**5**	]
2	[	1	3	2	**4**	**5**	]
	[	**1**	**2**	**3**	**4**	**5**	]

Ordenação Bolha - *Bubble Sort*

i = 4

j		1	2	3	4	5	
1	[	1	2	**3**	**4**	**5**	]

Ordenação Bolha - *Bubble Sort*

i = 4

j		1	2	3	4	5	
1	[	1	2	**3**	**4**	**5**	]
	[	**1**	**2**	**3**	**4**	**5**	]

Ordenação Bolha - *Bubble Sort*

Algoritmo em Java

```
1  public static void bolha(Item v[], int n) {
2    int i, j;
3    Item temp;
4    for (i = 1; i <= n - 1; i++) {
5      for (j = 1; j <= (n - i); j++) {
6        if (v[j].chave() > v[j + 1].chave()) {
7          temp = v[j];
8          v[j] = v[j + 1];
9          v[j + 1] = temp;
10       }
11     }
12   }
13 }
```

Ordenação Bolha - *Bubble Sort* - Análise de Complexidade

- Número de Comparações
  - Melhor Caso: n^2
  - Pior Caso: n^2
- Número Movimentações
  - Melhor Caso: 1
  - Pior Caso: n^2
- Vantagens
  - Algoritmo simples
  - Algoritmo estável
- Desvantagens
  - O fato de o arquivo já estar ordenado não ajuda em nada, pois o custo continua quadrático
- É possível fazer alguma melhoria???

Ordenação Bolha - *Bubble Sort*

```
1  public static void bolha(Item v[], int n) {
2    int i, j;
3    Item temp;
4    boolean trocou = false;
5    for (i = 1; i <= n - 1; i++) {
6      trocou = false;
7      for (j = 1; j <= n - i; j++) {
8        if (v[j].chave() > v[j + 1].chave()) {
9          temp = v[j];
10         v[j] = v[j + 1];
11         v[j + 1] = temp;
12         trocou = true;
13       }
14     }
15     if (!trocou) break;
16   }
17 }
```

Ordenação Bolha - *Bubble Sort* - Análise de Complexidade

- Número de Comparações
  - Melhor Caso: n
  - Pior Caso: n^2
- Número Movimentações
  - Melhor Caso: 1
  - Pior Caso: n^2

Ordenação por Seleção - *Selection sort*

- Um dos algoritmos mais simples de ordenação.
- Algoritmo:
  - **Selecione** o menor item do vetor.
  - Troque-o com o item da primeira posição do vetor.
  - Repita essas duas operações com os $n - 1$ itens restantes, depois com os $n - 2$ itens, até que reste apenas um elemento

Ordenação por Seleção - *Selection sort*

i = 1

j	Min		**1**	2	3	4	5	
2	1	[	3	<u>1</u>	5	4	2	]

Ordenação por Seleção - *Selection sort*

i = 1

j	Min		**1**	2	3	4	5	
2	1	[	3	<u>1</u>	5	4	2	]
3	2	[	3	1	<u>5</u>	4	2	]

Ordenação por Seleção - *Selection sort*

i = 1

j	Min		**1**	2	3	4	5	
2	1	[	3	$\underline{1}$	5	4	2	]
3	2	[	3	1	$\underline{5}$	4	2	]
4	2	[	3	1	5	$\underline{4}$	2	]

Ordenação por Seleção - *Selection sort*

i = 1

j	Min		**1**	2	3	4	5	
2	1	[	3	1	5	4	2	]
3	2	[	3	1	5	4	2	]
4	2	[	3	1	5	4	2	]
5	2	[	3	1	5	4	2	]

Ordenação por Seleção - *Selection sort*

i = 1

j	Min		**1**	2	3	4	5	
2	1	[	3	1	5	4	2	]
3	2	[	3	1	5	4	2	]
4	2	[	3	1	5	4	2	]
5	2	[	3	1	5	4	2	]
	2	[	**1**	**3**	**5**	**4**	**2**	]

Vetor[1] = 1
Vetor[2] = 3

Ordenação por Seleção - *Selection sort*

i = 2

j	Min		1	**2**	3	4	5	
3	2	[	**1**	3	<u>5</u>	4	2	]

Ordenação por Seleção - *Selection sort*

i = 2

j	Min		1	**2**	3	4	5	
3	2	[	**1**	3	$\underline{5}$	4	2	]
4	2	[	**1**	3	5	$\underline{4}$	2	]

Ordenação por Seleção - *Selection sort*

i = 2

j	Min		1	**2**	3	4	5	
3	2	[	**1**	3	<u>5</u>	4	2	]
4	2	[	**1**	3	5	<u>4</u>	2	]
5	2	[	**1**	3	5	4	<u>2</u>	]

Ordenação por Seleção - *Selection sort*

i = 2

j	Min		1	**2**	3	4	5	
3	2	[	**1**	3	5	4	2	]
4	2	[	**1**	3	5	4	2	]
5	2	[	**1**	3	5	4	2	]
	5	[	**1**	**2**	**5**	**4**	**3**	]

Vetor[2] = 2
Vetor[5] = 3

Ordenação por Seleção - *Selection sort*

i = 3

j	Min		1	2	**3**	4	5	
4	3	[	**1**	**2**	5	<u>4</u>	3	]

Ordenação por Seleção - *Selection sort*

i = 3

j	Min		1	2	**3**	4	5	
4	3	[	**1**	**2**	5	<u>4</u>	3	]
5	4	[	**1**	**2**	5	4	<u>3</u>	]

Ordenação por Seleção - *Selection sort*

i = 3

j	Min		1	2	**3**	4	5	
4	3	[	**1**	**2**	5	4	3	]
5	4	[	**1**	**2**	5	4	3	]
	5	[	**1**	**2**	**3**	**4**	**5**	]

Vetor[3] = 3
Vetor[5] = 5

Ordenação por Seleção - *Selection sort*

i = 4

j	Min		1	2	3	**4**	5	
5	4	[	**1**	**2**	**3**	4	5	]

Ordenação por Seleção - *Selection sort*

i = 4

j	Min		1	2	3	**4**	5	
5	4	[	**1**	**2**	**3**	4	5	]
	4	[	**1**	**2**	**3**	**4**	**5**	]

Vetor[4] = 4
Vetor[4] = 4

Ordenação por Seleção - *Selection sort*

Algoritmo em Java

```
1  public static void selecao(Item v[], int n) {
2    for (int i = 1; i <= n - 1; i++) {
3      int min = i;
4      for (int j = i + 1; j <= n; j++)
5        if (v[j].chave() < v[min].chave())
6          min = j;
7      Item x = v[min];
8      v[min] = v[i];
9      v[i] = x;
10   }
11 }
```

Ordenação por Seleção - *Selection sort*

- Número de Comparações
  - Melhor Caso: n^2
  - Pior Caso: n^2
- Número Movimentações
  - Melhor Caso: n
  - Pior Caso: n
- Vantagens:
  - Custo **linear** para o **número de movimentos** de registros.
  - É o algoritmo a ser utilizado para arquivos com registros muito grandes.
  - É muito interessante para arquivos pequenos.
- Desvantagens:
  - O fato de o arquivo já estar ordenado não ajuda em nada, pois o custo continua quadrático.
  - O algoritmo **não é estável**.

Ordenação por Seleção - *Selection sort*

- Invente um vetor-exemplo de entrada para demonstrar que ordenação por Seleção é instável. Mostre os passos da execução do algoritmo até que a estabilidade seja violada

Ordenação por Seleção - *Selection sort*

- Invente um vetor-exemplo de entrada para demonstrar que ordenação por Seleção é instável. Mostre os passos da execução do algoritmo até que a estabilidade seja violada

$$[\ 2\quad 2\quad 1\ ]$$

Ordenação por Seleção - *Selection sort*

- Invente um vetor-exemplo de entrada para demonstrar que ordenação por Seleção é instável. Mostre os passos da execução do algoritmo até que a estabilidade seja violada

$$\begin{array}{cccc} [& 2 & 2 & 1 &] \\ \hline [& 2 & \underline{2} & 1 &] \end{array}$$

Ordenação por Seleção - *Selection sort*

- Invente um vetor-exemplo de entrada para demonstrar que ordenação por Seleção é instável. Mostre os passos da execução do algoritmo até que a estabilidade seja violada

[	2	2	1	]
[	2	$\underline{2}$	1	]
[	2	2	$\underline{1}$	]

Ordenação por Seleção - *Selection sort*

- Invente um vetor-exemplo de entrada para demonstrar que ordenação por Seleção é instável. Mostre os passos da execução do algoritmo até que a estabilidade seja violada

$$
\begin{array}{cccc}
[& 2 & 2 & 1 &] \\
\hline
[& 2 & \underline{2} & 1 &] \\
[& 2 & 2 & \underline{1} &] \\
\hline
[& 1 & 2 & 2 &]
\end{array}
$$

Ordenação por Seleção - *Selection sort*

- Invente um vetor-exemplo de entrada para demonstrar que ordenação por Seleção é instável. Mostre os passos da execução do algoritmo até que a estabilidade seja violada

[	2	2	1	]
[	2	$\underline{2}$	1	]
[	2	2	$\underline{1}$	]
[	1	2	2	]
[	1	2	$\underline{2}$	]

Ordenação por Seleção - *Selection sort*

- Invente um vetor-exemplo de entrada para demonstrar que ordenação por Seleção é instável. Mostre os passos da execução do algoritmo até que a estabilidade seja violada

$$
\begin{array}{cccc}
[& 2 & 2 & 1 &] \\
\hline
[& 2 & \underline{2} & 1 &] \\
[& 2 & 2 & \underline{1} &] \\
\hline
[& 1 & 2 & 2 &] \\
[& 1 & 2 & \underline{2} &] \\
\hline
[& 1 & 2 & 2 &]
\end{array}
$$

Ordenação por Inserção - *Insertion sort*

- Método preferido dos jogadores de cartas.
  - As cartas são ordenadas da esquerda para direita uma por uma.
  - O jogador escolhe a segunda carta e verifica se ela deve ficar antes ou na posição que está.
  - Depois a terceira carta é classificada, deslocando-a até sua correta posição
  - O jogador realiza esse procedimento até ordenar todas as cartas
- Alto custo em remover uma carta de uma posição e colocá-la em outra quando a representação é por arranjos

Ordernação por Inserção - *Insertion sort*

i = 2

j	Sentinela[0]		1	**2**	3	4	5	
1	1	[	3	1	5	4	2	]

Ordernação por Inserção - *Insertion sort*

i = 2

j	Sentinela[0]		1	**2**	3	4	5	
1	<u>1</u>	[	<u>3</u>	1	5	4	2	]
	1	[	**1**	**3**	**5**	**4**	**2**	]

Ordernação por Inserção - *Insertion sort*

i = 3

j	Sentinela[0]		1	2	**3**	4	5	
2	<u>5</u>	[	1	<u>3</u>	5	4	2	]

Ordernação por Inserção - *Insertion sort*

i = 3

j	Sentinela[0]		1	2	**3**	4	5	
2	5	[	1	3	5	4	2	]
	5	[	**1**	**3**	**5**	**4**	**2**	]

Ordernação por Inserção - *Insertion sort*

i = 4

j	Sentinela[0]		1	2	3	**4**	5	
3	<u>4</u>	[	1	3	<u>5</u>	4	2	]

Ordernação por Inserção - *Insertion sort*

i = 4

j	Sentinela[0]		1	2	3	**4**	5	
3	<u>4</u>	[	1	3	<u>5</u>	4	2	]
2	<u>4</u>	[	1	<u>3</u>	4	5	2	]

Ordernação por Inserção - *Insertion sort*

i = 4

j	Sentinela[0]		1	2	3	**4**	5	
3	4	[	1	3	5	4	2	]
2	4	[	1	3	4	5	2	]
	4	[	**1**	**3**	**4**	**5**	**2**	]

Ordernação por Inserção - *Insertion sort*

i = 5

j	Sentinela[0]		1	2	3	4	**5**	
4	2	[	1	3	4	5	2	]

Ordernação por Inserção - *Insertion sort*

i = 5

j	Sentinela[0]		1	2	3	4	**5**	
4	2	[	1	3	4	5	2	]
3	2	[	1	3	4	2	5	]

Ordernação por Inserção - *Insertion sort*

i = 5

j	Sentinela[0]		1	2	3	4	**5**	
4	2	[	1	3	4	5	2	]
3	2	[	1	3	4	2	5	]
2	2	[	1	3	2	4	5	]

Ordernação por Inserção - *Insertion sort*

i = 5

j	Sentinela[0]		1	2	3	4	**5**	
4	2	[	1	3	4	5	2	]
3	2	[	1	3	4	2	5	]
2	2	[	1	3	2	4	5	]
	2	[	**1**	**2**	**3**	**4**	**5**	]

Ordernação por Inserção - *Insertion sort*

Algoritmo em Java

```
1  public static void insercao(Item v[], int n) {
2    int j;
3    for (int i = 2; i <= n; i++) {
4      Item x = v[i];
5      j = i - 1;
6      v[0] = x; /* sentinela */
7      while (x.chave() < v[j].chave()) {
8        v[j + 1] = v[j];
9        j--;
10     }
11     v[j + 1] = x;
12   }
13 }
```

Ordernação por Inserção - *Insertion sort*

- Número de Comparações
  - Melhor Caso: n
  - Pior Caso: n^2
- Número Movimentações
  - Melhor Caso: n
  - Pior Caso: n^2
- Características:
  - O número mínimo de comparações e movimentos ocorre quando os itens estão originalmente em ordem.
  - O número máximo ocorre quando os itens estão originalmente na ordem reversa.
  - É o método a ser utilizado quando o arquivo está “quase” ordenado.
  - É um bom método quando se deseja adicionar uns poucos itens a um arquivo ordenado, pois o custo é linear.
  - O algoritmo de ordenação por inserção é **estável**.

Exercício

- Dada a sequência de números: [3 4 9 2 5 1 8] Ordene em ordem crescente utilizando os três algoritmos aprendidos em sala (Bolha, Seleção e Inserção), apresentado a sequência dos números a cada passo.

Referência

ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007. 621 p.